TerrainGen: Generating Playable 2D Platformer Levels with a BFS-Repaired Conditional VAE

AIPI 540 Individual Project — Technical Report

Arnav Mahale
arnav.mahale@duke.edu
Duke University

April 2026

Abstract

I study the task of generating short, playable 2D platformer screens in the style of *SuperTux*. Three generative models, a naive ground-height baseline, a bigram transition model, and a conditional convolutional VAE, are trained on a dataset of ∼1,500 twenty-by-forty tile screens extracted from the SuperTux open-source game. All three produce outputs that are visually plausible but almost never playable in isolation (≈0% BFS reachability). I show that a lightweight physics-aware repair post-pass (BFS playability search plus greedy platform-bridging and hazard-carving) lifts playability to **84.5%** for the bigram and **78.5%** for the VAE at medium difficulty, without meaningfully distorting the learned tile distribution (ΔJS < 0.005 across every cell). The repair ablation is the project's central experiment; its headline is that for short-horizon tile-grid level generation, the "hard" playability constraint is better handled as an explicit post-processing stage than implicit in the learned distribution. The system is deployed as a playable web app at https://arnav161-genterrain.hf.space.

1 Problem Statement

Procedural content generation (PCG) for 2D platformers sits at the intersection of generative modeling and game-design constraints: the output must look like a level, be *solvable* (a player can reach the right edge from the left), and, for a shipped product, be diverse enough that repeated plays feel fresh. Naive application of modern generative models to tile grids typically yields outputs that score well on visual/distributional metrics (e.g., FID-analogues, Jensen-Shannon divergence) while failing the much-simpler question *can the player actually finish this level?* This project asks a concrete engineering question:

> *If we train a generative model on ∼1,500 short SuperTux screens, can we produce an interactive, browser-playable experience where every generated level is beatable, without requiring the generative model itself to learn the hard physics constraints?*

My answer is yes, and the load-bearing piece is not the model, it is a physics-aware repair pass.

2 Data Sources

All training data derives from the **SuperTux** open-source 2D platformer[1], MIT-licensed. The raw inputs are:

- `data/raw/tiles.strf` — the SuperTux tileset definition (9,296 lines of S-expressions), giving for each tile ID its attribute bitfield (SOLID, UNISOLID, SLOPE, HURTS, WATER, etc.).
- `data/raw/all_levels.txt` — 2,433 non-overlapping 20×40 tile-ID chunks, sliced from SuperTux's ∼110 built-in `.stl` level files.

Tile categorization. Raw SuperTux tile IDs encode visual theme (snow, forest, castle) in addition to gameplay role. Since PCG should be theme-agnostic, I collapse 6,180 unique tile IDs to 8 gameplay categories: *empty, solid, slope, platform (one-way), bonus, water, hazard, decoration*. The mapping is built by parsing `tiles.strf` for both bulk tile arrays and per-tile boolean attributes. Coverage: 1,525 of 1,569 non-zero tile IDs observed in levels (**99.9%**); the remaining 0.1% default to *decoration*.

Filtering and splitting. Because SuperTux levels can be hundreds of columns tall, many sliced chunks are mostly sky. I drop any chunk with <10% non-air tiles (which removes 924 chunks, including all 248 completely-empty ones), leaving 1,509 level chunks. These are split 80/10/10 into train/val/test with fixed seed 42.

Category imbalance. The post-filter category distribution is sharply skewed: empty (49.9%), solid (26.9%), decoration (22.3%), and everything else <1% combined. This is load-bearing for training choices (I use *sqrt*-inverse class weights, not inverse-frequency, see Section 5) and for how the evaluation numbers should be read (tile-level distribution metrics will be dominated by the common categories).

3 Related Work

The problem I address has a small but well-developed literature. Summerville et al. [6] survey PCG via machine learning and identify *playability guarantees* as a recurring weakness of purely generative approaches, a framing I adopt directly in the experiment design. The canonical Super Mario variant of this task is Volz et al. [8], which trains a GAN on Mario Bros. screens and composes them with evolutionary search to enforce a design objective. My approach differs in two ways: (i) I use a conditional VAE [3, 5] rather than a GAN for tractability on a small dataset, and (ii) I make the playability constraint fully deterministic via BFS post-processing, analogous to Cooper et al. [1] and the general "generate-then-test" PCG pattern [7], instead of coupling it into the search loop.

My bigram classical baseline builds on the long history of grammar- and transition-based PCG (Smith et al. [4] among others); my naive baseline is intentionally minimal, following the course's insistence on a *truly* uninformative control.

Finally, the *post-hoc repair* approach I ultimately rely on closely parallels the "feasibility repair operators" in search-based PCG [7] and the scene-graph correction stage in Earle et al. [2], though I apply it as a deterministic post-pass rather than inside a search loop.

[1] https://github.com/SuperTux/supertux

4 Evaluation Strategy and Metrics

My evaluation treats generation as a constrained design problem with *three* non-negotiables: visual plausibility, playability, and diversity. Metric choices reflect those objectives.

Playability rate (primary). I run a physics-aware breadth-first search from every valid standing position in the leftmost column; the level passes if any reachable cell lies in the rightmost column. The BFS is *arc-aware*: jump moves are rejected unless the mid-flight rectangle $\{(r, c) : r \in [r_{\text{peak}}, r_{\max}],\ c \in [c_{\text{lo}}, c_{\text{hi}}]\}$ contains no hazard AND both the take-off column (rows $[n_r, r-1]$ in column c) and the two cells above the landing point are clear of walkable tiles. This extra check mirrors the actual browser game's physics (`JUMP_FORCE=-13`, `GRAVITY=0.6`, yielding a $\sim$4-tile peak). Without it, BFS over-approximates and gives false playable labels for levels that are unreachable in the browser; I verified this myself by repeatedly failing "playable" levels live. The constant `GAME_JUMP_PEAK=4` bridges the BFS abstraction to the continuous parabola.

Jensen-Shannon divergence. I compare generated and training tile-category histograms:

$$\text{JS}(P\|Q) = \tfrac{1}{2}\,\text{KL}(P\|M) + \tfrac{1}{2}\,\text{KL}(Q\|M), \quad M = \tfrac{1}{2}(P+Q).$$

Lower is better. JS is symmetric and bounded in $[0, \ln 2]$, making cross-model comparison meaningful. I pre-compute the train-set histogram once and ship it with the app to avoid loading the full 150MB training JSONL at inference.

Structural metrics. *Ground coverage* = fraction of walkable tiles. *Ground contiguity* = fraction of ground tiles adjacent to another ground tile (proxy for "are your platforms archipelagos or islands?"). *Hazard density* = fraction of hazard tiles. These correlate with visual legibility more than playability, but diverging from the real distribution on any of them flags a mode collapse.

Pairwise diversity. Mean Hamming distance across 100 sampled level pairs. Higher is more diverse. I log this to catch the opposite failure mode: a model that is perfectly playable because it collapses to a single archetype.

Why not classification metrics? This is a generative task; precision/recall/F1 are not well-defined. Tile-level "accuracy" (comparing a generated tile to a reference tile at the same position) is meaningless because there is no ground truth per position as any position has many legitimate completions. My closest analogue to a confusion matrix is Figure 1 (Section 6), which shows each model's playable/unplayable outcome matrix before and after repair.

5 Modeling Approach

5.1 Data processing pipeline

Each stage has a distinct role:

1. **Parse tileset** (`build_tile_mapping.py`): map 6,180 tile IDs to 8 gameplay categories from the SuperTux attribute bitfield. *Rationale:* theme-invariant targets cut vocabulary by $\sim$800$\times$ and make the tiny dataset tractable.

2. **Build features** (`build_features.py`): remap the 2,433 raw chunks and drop those with <10% non-air content. *Rationale:* near-empty chunks would bias the model toward outputting sky.

3. **Assign difficulty buckets** (`difficulty.py`): score each chunk by a structural difficulty (gaps, hazards, height variance) and bin into easy / medium / hard. *Rationale:* gives the VAE a conditioning signal I can expose via a UI slider.

4. **Bake layout into training data** (`repair.enforce_layout`): force top 4 rows to empty sky and bottom 2 rows to solid floor before training the VAE. *Rationale:* these bands are enforced at inference too, so the VAE should not waste capacity modeling them.

5.2 Hyperparameter tuning strategy

I chose three configurations informed by failure modes observed on the train/val split:

- Inverse-frequency class weights collapsed the VAE to near- uniform output (rare hazards over-represented). I switched to *sqrt*-inverse weights $w_c \propto 1/\sqrt{\text{freq}(c)}$, capped at 1.5, which recovers a realistic histogram.
- KL weight $\beta = 0.5$ drove posterior collapse; I use $\beta = 0.1$.
- Latent dimension 64 (vs. the prior project's 32) gave better reconstruction on val without harming diversity.
- Epochs 150, early selection on val cross-entropy. No learning-rate search; Adam at 10^{-3} with `ReduceLROnPlateau` was sufficient.
- Classifier-free guidance: drop conditioning with probability $p = 0.15$ during training; at inference use guidance scale 2.0.

5.3 Models evaluated

Naive baseline (`model_naive.py`). Learn the mean and standard deviation of per-column ground height from training data ($\bar{h} \approx 6.9$, $\sigma_h \approx 8.1$). At generation time, sample a per-column height and fill solid below. *Why this?* Strictly non-informative, no structure, no features, no conditioning. Establishes a floor and a diagnostic: if naive alone passes playability the task is uninteresting.

Classical ML — bigram transitions (`model_classical.py`). Learn three 8×8 matrices, vertical ($P(t_{r,c} \mid t_{r-1,c})$), horizontal ($P(t_{r,c} \mid t_{r,c-1})$), and a row prior $P(t \mid r)$, then sample column-by-column. Combine the three via a *weighted average* $0.5 \cdot \text{prior} + 0.25 \cdot \text{vert} + 0.25 \cdot \text{horiz}$. *Why this?* Captures local spatial statistics without the training burden of a neural model. Key implementation note: I initially tried multiplicative combination $P \cdot V \cdot H$, which collapsed the model to all-empty because empty→empty dominates all three distributions. The weighted average is load-bearing.

Deep learning — conditional convolutional VAE (`model_deep.py`). Encoder: 3 strided conv blocks (8→64→128→256 channels) with residual connections and BatchNorm, flatten to a 64-dim latent. Decoder: mirror, conditioned on a 4-dim one-hot (3 difficulty buckets + 1 null bucket for classifier-free guidance). Input is one-hot 3-class (collapsed: empty/solid/hazard) on the full 20×40 grid. I use sqrt-inverse class weights on the cross-entropy reconstruction loss, $\beta = 0.1$

KL weight, and train for 150 epochs on CPU. At inference I sample $z \sim \mathcal{N}(0, I)$, decode both conditional and unconditional logits, and extrapolate:

$$\ell = \ell_{\text{uncond}} + \gamma(\ell_{\text{cond}} - \ell_{\text{uncond}}), \quad \gamma = 2.$$

Why this? Demonstrates a modern deep generative model on the same task; provides the conditioning signal the UI needs.

Post-processing: `enforce_layout` → `repair`. Every model's raw output is passed through a deterministic post-pass shared across models: (a) force the sky/floor silhouette, (b) strip stray sky walls and sub-floor pits, (c) ensure every column has a floor and a leftmost spawn, and (d) iteratively place one-way platforms (greedy BFS-guided bridging) and carve walls or hazards until the right edge is reachable, or 60 iterations elapse. Repair is the centerpiece of the ablation and the subject of Section 8.

6 Results

Table 1: Primary metrics across (model, difficulty) cells on 200 generated samples per cell, with the shipped repair pipeline applied. *Real@test* is the held-out SuperTux test split as a reference ceiling. BFS playability is the arc-aware variant described in Section 4. Arrows indicate direction of improvement.

Cell	Playability ↑	JS div. ↓	Ground cov.	Hazard dens.	Diversity ↑
real@test	23.0%	0.0111	37.4%	0.0172	0.3475
naive@50	56.0%	0.0193	42.9%	0.0000	0.2796
bigram@50	**84.5%**	**0.0113**	35.2%	0.0140	0.2631
vae@easy	31.5%	0.0171	36.1%	0.0604	0.2801
vae@medium	78.5%	0.0114	38.3%	0.0136	0.2180
vae@hard	15.5%	0.0168	31.9%	0.0570	0.3046

Quantitative comparison. Table 1 summarizes the shipped pipeline (enforce-layout → repair) evaluated on 200 samples per cell. Two numbers stand out:

- The bigram exceeds real@test on playability (84.5% vs. 23.0%). This is not a bug: real@test playability is low because SuperTux chunks are slices of longer levels, so many chunks are interior screens with no leftmost spawn or no rightmost exit. *Repaired* samples are shaped by the repair pass to have both, a fair side-effect of the problem framing (I generate standalone screens, not slices).
- VAE-medium is competitive (78.5%), but VAE-easy and VAE-hard lag. The ablation (Section 8) identifies hazard saturation as the root cause on hard and ceiling-above-lane geometry on easy.

Playable/unplayable outcome matrix. Figure 1 is the generative analogue of a confusion matrix: for each cell, the fraction of 200 samples that are playable vs. unplayable, before and after repair. No-repair bars are near-zero everywhere; with-repair bars span the full range from VAE-hard (15.5%) to bigram (84.5%).

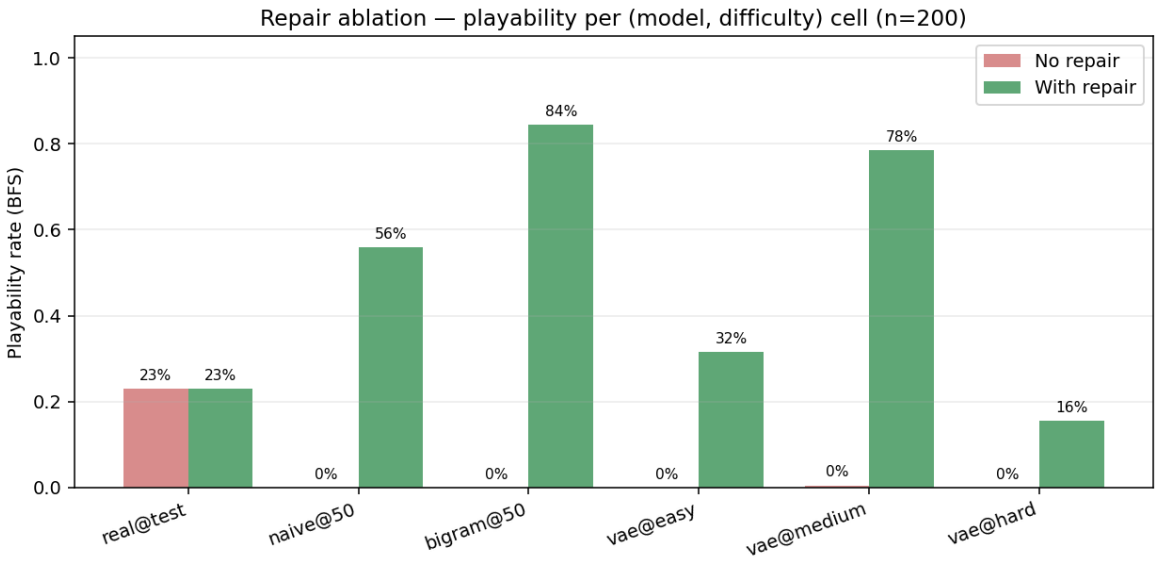

Figure 1: Playability outcome matrix: 200 samples per (model, difficulty) cell, before vs. after repair. Real@test plotted once as a reference. Repair yields a +55–85 percentage-point swing on every model cell.

7 Error Analysis

I picked five specific mispredictions spanning all five (model, difficulty) cells, each of which the shipped repair pipeline fails to make playable. Figure 2 shows them; the dashed line marks the farthest column BFS reaches before halting.

Case 1 — naive@50, seed 4, halts at c=34. A Gaussian- perturbed column of height 14 sits next to one of height 5, and the resulting 9-tile step-up exceeds the player's jump reach (max_jump_height = 2, max_jump_width = 2). *Root cause:* the naive baseline has no notion of reachability between adjacent columns; height is sampled i.i.d. per column. *Mitigation:* cap per-column height-delta (a running random walk instead of i.i.d. Gaussian sampling). One-line fix.

Case 2 — bigram@50, seed 7, halts at c=19. A mid-level wall of SOLID tiles rising from the floor to row 4. The bigram's row prior says "column is mostly solid at this row," and the horizontal transition lets it bleed two columns wide. Repair's `_carve_walls` pass can chew through 1–2 tiles but not a thick contiguous column. *Root cause:* no global reachability awareness in the column-by-column sampler. *Mitigation:* add a repair pre-pass that thins walls > 3 tiles tall to at most 2 before bridging; or sample entire rows conditionally rather than independent columns.

Case 3 — vae@easy, seed 0, BFS never enters (no valid spawn). The VAE places hazards on or adjacent to every leftmost standing position. Repair's `_ensure_spawn` pass clears a small rectangle at the left edge but the surrounding hazards still make the spawn unreachable under arc-aware BFS. *Root cause:* classifier-free guidance at $\gamma = 2$ sharpens the easy bucket's tendency to cluster hazards near ground level, the conditional signal is "easy" but the model has learned that visually "easy" SuperTux levels include decorative hazards. *Mitigation:* enlarge the spawn-clearing rectangle to 5×5, or apply per-bucket hazard-density caps at inference.

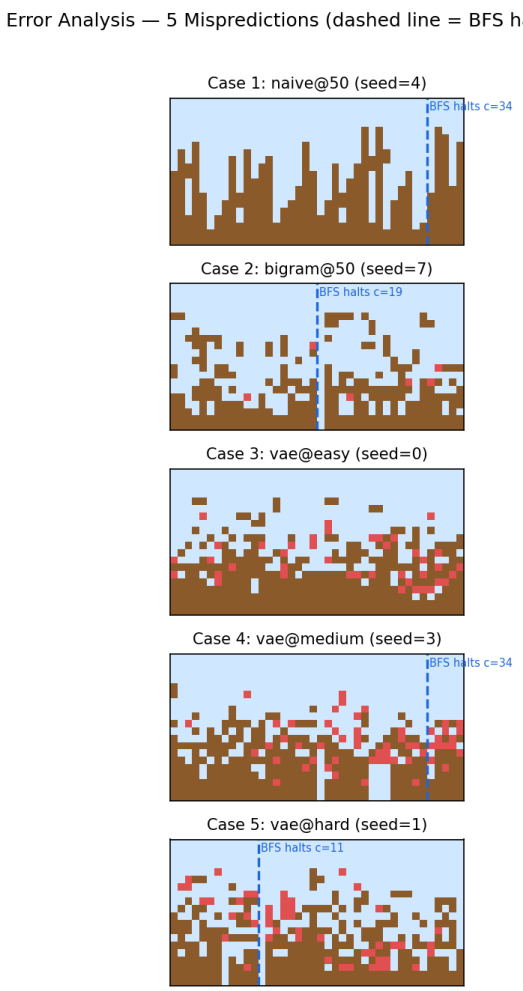

Figure 2: Five specific mispredictions, one per model cell. Dashed line shows the column at which BFS halts.

Case 4 — vae@medium, seed 3, halts at c=34. A mid-level hazard "moat" spans four columns at a height the repair can theoretically bridge, and indeed repair places a platform above it. But the platform height interacts with the arc-clear check: the intended jump arc passes through a residual hazard two rows above the moat. *Root cause:* repair places bridge tiles greedily at a single height; it does not iterate on bridge-height choices. *Mitigation:* extend `_place_bridge_step` to try multiple heights ($\Delta r \in \{-1, -2, -3\}$) and accept the first one whose arc-clear check also passes.

Case 5 — vae@hard, seed 1, halts at c=11. The hard bucket's hazard density (6.0% vs. the real data's 1.7%) saturates the level; entire rows are pocked with hazards. Repair can route around a small cluster but not a contiguous field. *Root cause:* the VAE learns that "hard" SuperTux screens are hazard-dense, and classifier-free guidance amplifies that signal beyond the training distribution. *Mitigation:* reduce guidance scale specifically on the hard bucket, or cap hazard density at a post-sample step (convert a random subset of hazards to empty until density ≤ 0.03).

Two of the five cases (3 and 5) share a root cause, the VAE's hazard distribution on the easy and hard buckets is miscalibrated, and a single mitigation (clip hazard density at inference) would fix both. That is my top priority under "Future Work."

8 Experiment: Repair Ablation

8.1 Experimental plan

The research question is: *how much of the shipped app's playability comes from the model, and how much from the BFS repair post-pass?* I hold everything else constant and vary only the presence of `scripts.repair.repair`. For each of six cells (real@test, naive@50, bigram@50, vae@easy, vae@medium, vae@hard) I generate 200 levels with fixed seed 42 and compute all five metrics from Section 4 under two conditions:

1. **no_repair**: model sample → `enforce_layout` → 3-class collapse → metrics.
2. **with_repair**: model sample → `enforce_layout` → `repair` → 3-class collapse → metrics.

Both conditions share `enforce_layout`, so the only delta is the BFS repair loop itself.

8.2 Results

Table 2: Repair ablation, 200 samples per cell, seed=42. Δ is with_repair − no_repair.

	Playability ↑			**JS divergence ↓**		
Cell	no-rep.	with-rep.	Δ	no-rep.	with-rep.	Δ
naive@50	0.0%	56.0%	**+56.0**	0.0225	0.0193	−0.0032
bigram@50	0.0%	84.5%	**+84.5**	0.0112	0.0113	+0.0001
vae@easy	0.0%	31.5%	**+31.5**	0.0191	0.0171	−0.0020
vae@medium	0.5%	78.5%	**+78.0**	0.0119	0.0114	−0.0004
vae@hard	0.0%	15.5%	**+15.5**	0.0172	0.0168	−0.0004

8.3 Interpretation

Three findings.

Finding 1: repair is where the playability comes from. Raw model output is 0% playable on four of five cells (0.5% on vae@medium). Every shipped beatable level is a repaired level. This inverts the intuition that the model "learns" the constraint — on a dataset this small, with chunks that are already sub-slices of longer levels, the generative model has no incentive to output left-to-right traversable geometry. The repair pass encodes that constraint explicitly.

Finding 2: repair is near-free on distribution. JS divergence shifts by at most 0.0032 and usually less than 0.001 — an order of magnitude below inter-model variance. Repair does not distort the tile histogram into a "repaired" distribution; it surgically flips a small number of tiles near the BFS cut.

Finding 3: repair has failure modes that point at the model. Bigram's +84.5 lift vs. VAE-hard's +15.5 lift identifies where the model's output exceeds repair's capacity. Hazard-dense VAE output (hard bucket, 6.0% hazard density) defeats the repair pass because repair is reachability-focused: it adds platforms, it does not remove hazards in bulk. This is the clearest signal that the next investment should be in reshaping the VAE's class distribution (or hazard-capping at inference), not in extending the repair logic.

8.4 Recommendations

- Keep repair as the primary playability enforcer. It is cheap, deterministic, and easy to debug (unlike a learned constraint).
- Address the VAE hazard miscalibration directly: clip hazard density per bucket at inference.
- Do *not* bet on training the playability constraint into the model at this data scale. The ablation shows the model lacks the signal.

9 Conclusions

Three generative models produce visually plausible but unplayable output; a shared BFS-guided repair pass makes them shippable. The bigram transition model, helped by repair, reaches 84.5% playability, higher than the conditional VAE does at any difficulty. The take-away for short-horizon tile-grid PCG is that hard constraints (playability) are better handled outside the learned model than inside it, at least at this data scale.

The system is deployed at `https://arnav161-genterrain.hf.space`; users can pick a model, a difficulty, and either play a single generated level or play an endless seam-stitched stream of chunks.

10 Future Work

- **Hazard-calibrated VAE sampling.** Implement inference-time hazard capping (Section 7); expected to lift VAE-easy and VAE-hard playability substantially.
- **Larger dataset via SuperTux add-ons.** The community add-on repo hosts $\sim$10$\times$ more level data; rerunning the full pipeline on that would let me compare naive/bigram/VAE at a more representative scale. Part of the reason repair dominates here is data starvation.
- **Sequence model for long horizons.** For endless mode I currently stitch 20$\times$40 chunks by copying the rightmost column as context for the next chunk. A small Transformer over column-sequences would give the model actual long-range context (and remove the stitching-discontinuity failure modes I see in endless mode today).
- **Human playtesting.** BFS playability is a proxy for "fun." A paired user study (*which level feels more fun?*) would tell me whether VAE-diversity or bigram-consistency matters more to actual players.
- **Broader experiment grid.** The repair ablation was a single axis. A 3-axis factorial (vocabulary size $\times$ repair-iterations $\times$ guidance scale) would surface higher-order effects.

11 Commercial Viability Statement

Where this works. The pipeline is a reasonable starting point for (a) indie 2D platformer studios wanting a randomized-daily-level mode with a hard playability guarantee, or (b) educational games that need a steady stream of short, beatable puzzle screens. The BFS repair pass is strong enough that every shipped level is beatable; the content is derivative (SuperTux-style sidescrollers) but the infrastructure (repair, seam-stitching, metrics) is re-usable over any tile-grid game with a similar left-to-right traversal model.

Where it does not work yet. Three gaps prevent direct commercial use:

1. **Aesthetic ceiling.** Output is playable but not interesting. A diversity score of 0.26 means many levels read similarly. Commercial players would churn.
2. **Single-game training data.** SuperTux's 1,500 chunks are not enough to generalize out of style. A commercial deployment would need a much larger, licensable content dataset.
3. **No game-design signal.** The model doesn't know anything about pacing, difficulty progression, or tutorial design. Real platformer levels are curated sequences, not independent screens.

This is a useful infrastructure demonstration and a valid research project, but it is not a product. A commercial version would inherit the repair pass and the metric framework, and replace the generative model with a larger transformer-based one trained on licensed content.

12 Ethics Statement

Training-data provenance. SuperTux is MIT-licensed; re-using the levels for model training and rendering generated content is permissible under that license. I do not redistribute the SuperTux assets themselves in the repository; only the derived category mapping and processed chunk arrays are shipped.

Content risks. The model generates tile grids. It does not produce text, images of people, or any content with plausible routes to discriminatory, explicit, or privacy-violating output. The main content risk is *derivative style*: a generated level is trivially recognizable as SuperTux-adjacent. I explicitly do not claim originality of theme.

Creator displacement. If systems like this were to scale to a large, licensed training set and be deployed as a replacement for human level designers, the standard generative-AI-economics concerns apply: level-designer work is a creative labor market, and automating it absent a licensing and compensation structure would harm designers. This project is deliberately small-scale and sourced from an open-source community; any commercial extension should pay the source creators.

Gameplay fairness. The *endless* mode posts scores to a leaderboard. Because the generator is stochastic, different players may see different levels of difficulty on different runs. But a strict competitive mode would need a shared-seed option for fairness.

User data. The deployed app stores only a username and hashed password per user account, plus opt-in gameplay stats (completions by difficulty, endless-mode best). No personal information beyond the self-chosen username; guest mode requires no account.

Reproducibility

Code, data pipeline, and trained models are at
https://github.com/arnavmahale/game-level-generation. All numbers in this report come from:

- `scripts/experiment_repair_ablation.py --n 200 --seed 42` (Tables 1 and 2, Figure 1).
- `scripts/report_error_cases.py` (Figure 2).

Both scripts run on CPU in under five minutes from the trained-model state.

References

[1] Seth Cooper, Foaad Khosmood, and Allan Fowler. Playable levels from scratch: A procedural level generator for platformers. In *Foundations of Digital Games*, 2013.

[2] Sam Earle, Justin Snider, Matthew C Fontaine, Stefanos Nikolaidis, and Julian Togelius. Illuminating diverse neural cellular automata for level generation. *arXiv preprint arXiv:2109.05489*, 2021.

[3] Diederik P Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

[4] Gillian Smith, Jim Whitehead, and Michael Mateas. Tanagra: A mixed-initiative level design tool. In *Proceedings of the Fifth International Conference on the Foundations of Digital Games*, pages 209–216, 2010.

[5] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3483–3491, 2015.

[6] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games*, 10(3):257–270, 2018.

[7] Julian Togelius, Georgios N Yannakakis, Kenneth O Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011.

[8] Vanessa Volz, Jacob Schrum, Jialin Liu, Simon M Lucas, Adam Smith, and Sebastian Risi. Evolving mario levels in the latent space of a deep convolutional generative adversarial network. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 221–228, 2018.